C Programming Assignment
Questions 1–12

General note: Compile each program separately because every question contains its own `main()` function.

Q1. Sum of the series: 1 + 1/4 + 1/9 + 1/16 + ...

```
#include <stdio.h>

int main()
{
    int i, n;
    double sum = 0.0;

    printf("Enter the number of terms: ");
    scanf("%d", &n);

    for(i = 1; i <= n; i++)
    {
        sum = sum + 1.0 / (i * i);
    }

    printf("Sum = %.5lf", sum);

    return 0;
}
```

Note: Note: This program calculates the sum of the first n terms. Since the series converges slowly, use a sufficiently large n to obtain a close approximation to the infinite sum.

Q2. Sum of the alternating series: 1 - 1/4 + 1/9 - 1/16 + ...

```
#include <stdio.h>
#include <math.h>

int main()
{
    int i, n;
    double sum = 0.0;
```

```
    printf("Enter the number of terms: ");
    scanf("%d", &n);

    for(i = 1; i <= n; i++)
    {
        sum = sum + pow(-1, i + 1) / (i * i);
    }

    printf("Sum = %.5lf", sum);

    return 0;
}
```

Note: Note: Enter a sufficiently large number of terms for a close approximation to the infinite series.

Q3. Sum of the series: 1 + 1/2! + 1/3! + 1/4! + ...

```
#include <stdio.h>

int main()
{
    int i;
    double sum = 1.0, fact = 1.0, term;

    for(i = 1; ; i++)
    {
        fact = fact * i;
        term = 1.0 / fact;

        if(term < 0.000001)
            break;

        sum = sum + term;
    }

    printf("Sum = %.5lf", sum);

    return 0;
}
```

Note: The series is the expansion of e. The small-term condition stops the calculation when further terms have a negligible effect at the requested precision.

Q4. Sum of the series for e^(-x)

```
#include <stdio.h>

int main()
{
    int i;
    double x, sum = 1.0, term = 1.0;

    printf("Enter the value of x: ");
    scanf("%lf", &x);

    for(i = 1; ; i++)
    {
        term = term * (-x) / i;

        if(term > -0.000001 && term < 0.000001)
            break;

        sum = sum + term;
    }

    printf("Sum = %.5lf", sum);

    return 0;
}
```

Note: Series used: e^(-x) = 1 - x + x^2/2! - x^3/3! + ... The recurrence calculates each new term from the previous one.

Q5. Calculate ln(0.9) using the series for ln(1 - x)

```
#include <stdio.h>

int main()
{
    int i;
    double x = 0.1, power = 1.0, sum = 0.0, term;

    for(i = 1; ; i++)
    {
        power = power * x;
        term = power / i;
        sum = sum + term;

        if(term < 0.000001)
```

```
            break;
    }

    printf("ln(0.9) = -%.5lf", sum);

    return 0;
}
```

Note: Since 0.9 = 1 - 0.1, take x = 0.1. The identity is ln(1 - x) = -(x + xˆ2/2 + xˆ3/3 + ...).

Q6. Calculate ln(1.1) using the series for ln(1 + x)

```
#include <stdio.h>

int main()
{
    int i;
    double x = 0.1, power = 1.0, sum = 0.0, term;

    for(i = 1; ; i++)
    {
        power = power * x;
        term = power / i;

        if(i % 2 == 1)
            sum = sum + term;
        else
            sum = sum - term;

        if(term < 0.000001)
            break;
    }

    printf("ln(1.1) = %.5lf", sum);

    return 0;
}
```

Note: Since 1.1 = 1 + 0.1, take x = 0.1. The identity is ln(1 + x) = x - xˆ2/2 + xˆ3/3 - ... .

Q7. Generate the first 50 terms of the Fibonacci series (five per line)

```
#include <stdio.h>

int main()
{
    long long a = 0, b = 1, c;
    int i;

    for(i = 1; i <= 50; i++)
    {
        printf("%lld\t", a);

        c = a + b;
        a = b;
        b = c;

        if(i % 5 == 0)
            printf("\n");
    }

    return 0;
}
```

Note: long long is used instead of int because the later Fibonacci terms are too large for a typical 32-bit int.

Q8. Generate the first 50 prime numbers (five per line)

```
#include <stdio.h>

int main()
{
    int n = 2, i, count = 0, flag;

    while(count < 50)
    {
        flag = 0;

        for(i = 2; i * i <= n; i++)
        {
            if(n % i == 0)
            {
```

```
                flag = 1;
                break;
            }
        }

        if(flag == 0)
        {
            printf("%d\t", n);
            count++;

            if(count % 5 == 0)
                printf("\n");
        }

        n++;
    }

    return 0;
}
```

Note: A number is prime if it has no positive divisors other than 1 and itself. The flag becomes 1 if a divisor is found.

Q9. Generate the first 15 unique Pythagorean triplets

```
#include <stdio.h>

int main()
{
    int a, b, c, count = 0;

    for(a = 1; count < 15; a++)
    {
        for(b = a + 1; b <= 100 && count < 15; b++)
        {
            for(c = b + 1; c <= a + b && count < 15; c++)
            {
                if(a*a + b*b == c*c)
                {
                    printf("%d\t%d\t%d\n", a, b, c);
                    count++;
                }
            }
        }
```

```
    }

    return 0;
}
```

Note: The condition a < b < c ensures that a triplet is printed in only one order. A Pythagorean triplet satisfies a^2 + b^2 = c^2.

Q10. Arrange numbers in ascending order using Bubble Sort

```
#include <stdio.h>

int main()
{
    int a[100], n, i, j, temp;

    printf("Enter the number of terms (1 to 100): ");
    scanf("%d", &n);

    if(n < 1 || n > 100)
    {
        printf("Invalid number of terms.");
        return 0;
    }

    printf("Enter the numbers:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    for(i = 0; i < n - 1; i++)
    {
        for(j = 0; j < n - i - 1; j++)
        {
            if(a[j] > a[j + 1])
            {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }

    printf("Numbers in ascending order:\n");
    for(i = 0; i < n; i++)
```

```
        printf("%d\t", a[i]);

    return 0;
}
```

Note: Bubble Sort compares adjacent elements and swaps them if they are in the wrong order. After each pass, the largest unsorted element reaches the end.

Q11. Arrange numbers in descending order using Quick Sort

```
#include <stdio.h>

void quicksort(int a[], int low, int high)
{
    int i = low, j = high;
    int pivot = a[(low + high) / 2];
    int temp;

    while(i <= j)
    {
        while(a[i] > pivot)
            i++;

        while(a[j] < pivot)
            j--;

        if(i <= j)
        {
            temp = a[i];
            a[i] = a[j];
            a[j] = temp;
            i++;
            j--;
        }
    }

    if(low < j)
        quicksort(a, low, j);

    if(i < high)
        quicksort(a, i, high);
}

int main()
```

```
{
    int a[100], n, i;

    printf("Enter number of terms (1 to 100): ");
    scanf("%d", &n);

    if(n < 1 || n > 100)
    {
        printf("Invalid number of terms.");
        return 0;
    }

    printf("Enter the numbers:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    quicksort(a, 0, n - 1);

    printf("Numbers in descending order:\n");
    for(i = 0; i < n; i++)
        printf("%d\t", a[i]);

    return 0;
}
```

Note: Quick Sort uses divide and conquer. This version places larger values before smaller values and uses the middle element as the pivot.

Q12. Calculate factorial using a function and evaluate the binomial coefficient nCr

```
#include <stdio.h>

long long factorial(int n)
{
    int i;
    long long fact = 1;

    for(i = 1; i <= n; i++)
        fact = fact * i;

    return fact;
}

int main()
```

```
{
    int n, r;
    long long result;

    printf("Enter n: ");
    scanf("%d", &n);

    printf("Enter r: ");
    scanf("%d", &r);

    if(n < 0 || r < 0 || r > n || n > 20)
    {
        printf("Enter values with 0 <= r <= n <= 20.");
        return 0;
    }

    result = factorial(n) / (factorial(r) * factorial(n - r));

    printf("Binomial coefficient = %lld", result);

    return 0;
}
```

Note: The formula is nCr = n! / [r!(n-r)!]. The program restricts n to 20 so that factorial values fit in a signed long long integer.